AI VIDEO STUDIO · PRODUCTION DOCUMENT

AI Pipeline Spec

The build-ready contract for the ten-stage pipeline: topic to publish pack. Every stage, its inputs and outputs, its model and tool choices, and the gates a video must pass before it renders.

Product
AI Video Studio - web app, Windows + Mac

Stack
Next.js, FastAPI, Redis / Celery, faster-whisper, Remotion, ffmpeg, R2 / S3

Route
Clean-room build. Spike 5-7 days, MVP 6-8 weeks, sellable v1 +4-6 weeks

Pricing model
Credits / subscription. Not one-time - the product carries render and generation costs on every video

Output, MVP
1080p MP4 + publish pack. 4K is post-MVP

Asset default
Licensed stock (Pexels / Pixabay). YouTube clips are opt-in only - see Section 6

How to read this document

Sections 1-2 define the shared contract and research stage every engineer builds against. Sections 3-8 specify the remaining stages, in pipeline order. Section 9 defines the quality gates. Section 10 covers cost, run order, and the one decision left open. Where a number is a third-party price, it is labelled an estimate or a range - verify it at procurement, do not quote it to customers.

Clean-room rule, applied throughout: no code, text, or design is copied from Slot Studio, OpenChatCut, Palmier Pro, or Pallaidium. Patterns only - editable timeline, proposal-based editing, swappable generation backend - re-implemented here in our own words and code.

1. Pipeline contract - one shape for every stage

FastAPI is the only orchestrator. The Next.js frontend never calls a worker, a model, or storage directly. Each stage runs as a Celery task on Redis, reads a versioned job manifest from R2 / S3, and writes its outputs back to the same manifest. A stage can be re-run without re-running the stages before it.

1 Topic | 2 Research | 3 Script | 4 Voice | 5 Cleanup | 6 Visual plan | 7 Generation | 8 Motion graphics | 9 Timeline edit | 10 Render + pack

Manifest field	Rule
`project_id, stage, stage_version`	Every artifact carries all three. Re-running a stage bumps its version; downstream stages pin the version they consumed.
`inputs[], outputs[]`	R2 / S3 keys plus SHA-256, byte size, and media type. No signed URLs stored in the manifest - they are minted on read.
`model, provider, cost_estimate`	Recorded per generation or inference call, so credit metering and the cost report in Section 10 come from the manifest, not from logs.
`rights[]`	One entry per external asset: source, licence type, and the opt-in flag where it applies. The render gate reads this array - see Section 9.
`status, error`	`queued | running | needs_review | passed | failed`. A failed stage returns a typed error code, never a free-text exception, so the UI can offer a retry that makes sense.

Idempotency and retries

- Every Celery task is idempotent on `(project_id, stage, input_hash)`. A duplicate delivery returns the existing output instead of billing a second generation.
- Transient provider errors retry twice with backoff. Content or validation errors do not retry - they go to `needs_review` for the user.
- Long stages (cleanup, generation, render) emit progress events over Redis pub/sub; the frontend shows stage, percent, and elapsed time, never a bare spinner.
- Intermediate files live under `projects/{project_id}/stages/{stage}/v{n}/`. Nothing is deleted until the project is exported and 7 days have passed.

Language and format defaults

Pipeline language is set per project (default English; Hinglish voice and captions are supported by the same stages, not a separate path). All text artifacts are UTF-8 JSON. All audio intermediates are 48 kHz WAV; all video intermediates are the Remotion composition itself - no flattened video exists until render.

2. Research - a cited brief, or no script

Input: topic string, target length (short under 90 seconds, standard 4-8 minutes, long 9-15 minutes), audience note, and language. **Output:** `research.json` - claims, sources, and a suggested angle.

Field	Rule
`claims[]`	Each claim: text, at least one source URL, publisher name, and the date the source was published or accessed. A claim with no source is dropped, not flagged.
`facts_to_avoid[]`	Contested, outdated, or single-source claims the script must not state as fact. The script stage receives this list explicitly.
`angle`	One recommended framing in two sentences, plus two alternatives. The user picks or edits before scripting - this is the cheapest review point in the pipeline.
`assets_hint[]`	Search terms for stock and generation, collected during research so the visual plan does not start cold.

Method

- The research worker queries web search, fetches the top sources, and extracts claims with an LLM call constrained to return only statements supported by a fetched page. No claim is generated from model memory alone.
- Duplicate and near-duplicate sources are merged. Paywalled or unreachable sources are recorded as `unverified` and cannot support a claim.
- Numbers get extra handling: any statistic, price, or date in a claim must appear verbatim in its source text. The worker checks this by string match before the claim is accepted.
- Research is cached by normalised topic for 24 hours inside a project only. It is never shared across users - topics are user data.

> **Gate:** the user approves the brief, or edits the angle, before the script stage can run. Skipping approval is a per-project setting, off by default for the first three projects a user creates.

3. Script - scene JSON, not a blob of text

Input: approved `research.json`. **Output:** `script.json` - an ordered scene list the rest of the pipeline can time, voice, and visualise scene by scene.

```
{
  "scenes": [{
    "id": "s03", "narration": "...",
    "on_screen_text": "...", "visual_intent": "chart | stock | generated | presenter",
    "claim_refs": ["c07"], "est_seconds": 11.5
  }],
  "hook": "first-3-seconds line", "cta": "..."
}
```

- **Word budget:** narration is written to 140-155 words per minute of target length. Estimated scene duration is words divided by that rate; it is replaced by real timings after voice (Section 4).
- **Hook rule:** scene 1 states the payoff or the tension in the first 3 seconds. No channel intro, no "in this video we will" before the hook.
- **Citation rule:** a scene that states a researched fact must carry its `claim_refs`. Scenes with no refs are treated as opinion or transition and must not introduce new numbers.
- **Structure presets:** explainer, list / tier, story, comparison, and news recap. A preset sets scene rhythm and which motion-graphic templates are likely - it does not change the schema.
- **Editing:** the user edits narration in the UI as text; the JSON regenerates timings on save. Find-and-replace across scenes and a read-aloud duration estimate are in the MVP editor. AI rewrite acts on one scene at a time, so a rewrite can never silently change the rest of the script.

4. Voice - recorded or AI, one timing source

Both voice paths converge on the same artifact: `voice.wav` plus `words.json` - word-level timings from faster-whisper. Every downstream stage (cleanup, captions, visual sync, timeline) reads `words.json`, never the raw audio.

Path A - user records

- In-browser recording per scene or as one take, with a level meter and a 3-second test playback before the full take. Upload of WAV / MP3 / M4A is also accepted.
- Recording is stored raw and never overwritten. Cleanup (Section 5) works on a copy.
- faster-whisper transcribes the raw take with word timestamps and language detection. If detected language differs from the project language, the user is told before cleanup runs.

Path B - text-to-speech

- Narration is synthesised per scene, so a script edit re-synthesises only the changed scenes. Per-scene audio is concatenated with 250 ms of silence at scene joins.
- Voice catalogue: a small set of licensed stock voices per language at MVP, each with a fixed provider voice ID recorded in the manifest. Users can preview 10 seconds before committing credits to a full synthesis.
- Pronunciation dictionary per project: user-supplied word-to-phonetic or word-to-spelling entries (names, brands, acronyms) applied before synthesis and before whisper alignment.
- SSML is not exposed to users. Emphasis comes from punctuation and an optional per-scene delivery note (`calm | energetic | serious`) mapped to provider parameters where supported.

Voice cloning

- Cloning is consent-gated: the user records a spoken consent statement plus the provider's minimum sample, in the same session. The consent recording and timestamp are stored with the voice profile and are never used as training audio.
- A cloned voice belongs to one account, is never listed in the shared catalogue, and is deleted - profile and provider-side copy - within 24 hours of a delete request.
- Cloned voices carry a disclosure flag in the manifest. The publish pack (Section 8) includes the platform AI-content disclosure note when the flag is set.
- Cloning a voice the user cannot verify as their own is out of scope for MVP and v1. No celebrity or third-party voice profiles are offered.

Provider choice for TTS and cloning is the one open decision in this document - see Section 10. The pipeline contract above is provider-neutral on purpose: scenes in, per-scene WAV out, cost recorded per call.

5. Voice cleanup - breaths and fillers out, words untouched

> **The 0-words-clipped rule:** cleanup may remove silence, breaths, and filler sounds. It must never cut, shorten, or overlap a spoken word. This rule outranks loudness, pacing, and file-size targets. Any edit that touches a word boundary is rejected automatically.

Input: raw `voice.wav` + `words.json`. **Output:** `voice_clean.wav`, an edit decision list (`cleanup.edl.json`), and a verification report. The raw file is retained.

Pass	What it does	Guardrail
Silence trim	Pauses over 700 ms are shortened to 350-450 ms. Leading and trailing silence over 300 ms is removed.	Cuts land at least 80 ms outside the nearest word start or end in `words.json`.
Breath removal	Breath events detected between words are attenuated by 12-18 dB, not hard-cut, so phrasing still sounds human.	Only in gaps of 120 ms or more with no word energy. Attenuation, never deletion.
Filler removal	Standalone fillers matched against the transcript (*um, uh, er* and language equivalents) are cut when they form their own token with silence or a breath on at least one side.	A filler inside a word, or a word that merely sounds like a filler, is kept. Uncertain cases are listed for the user, not cut.
Level and noise	ffmpeg loudnorm to -14 LUFS integrated, true peak -1 dBTP for the final mix; light noise reduction on recorded voice only.	Noise reduction strength is capped so sibilants survive. TTS audio skips this pass.

Verification - how the rule is enforced, not promised

1. Re-transcribe `voice_clean.wav` with faster-whisper.
2. Diff the cleaned transcript against the original word list. Any missing, merged, or changed word fails the stage.
3. On failure, cleanup retries once with conservative settings (silence trim only). A second failure sends the raw audio forward, marks the stage `needs_review`, and tells the user cleanup was skipped - it never ships a clipped voice silently.
4. The user can audition any single edit from the EDL and restore it. Restores update the EDL; the audio is re-rendered from raw + EDL, never edited destructively.

Visual plan - decided here, executed in Section 6

With real word timings known, the visual planner assigns each scene a visual: `stock | generated | motion_graphic | presenter / avatar | screen`. It prefers, in order: a template when the scene carries data or a ranking, licensed stock when a real-world subject is named, and generation only when neither fits. Each assignment records its reason in `visual_plan.json`, so a user asking "why this clip" gets an answer, not a shrug.

6. Generation backend - one contract, swappable adapters

Generation-into-project means generated media lands in the project as a normal asset - R2 / S3 key, rights entry, scene assignment - not as a chat attachment the timeline cannot reference. All generation goes through one internal contract modelled on the OpenAI API shape, so swapping a provider changes an adapter, not the pipeline.

```
POST /v1/generations        { kind: image|video, prompt, scene_id,
                              duration_s, aspect, seed?, refs[] }
GET  /v1/generations/{id}    { status, outputs[], cost_estimate, provider }
POST /v1/generations/{id}/cancel
```

Adapter	Role	Notes
Hosted API adapter (fal-type)	Default for MVP image and short video generation.	Per-call pricing, no GPU to run. Costs are estimates / ranges - see Section 10. Jobs are polled, with a 10-minute hard timeout per clip.
ComfyUI adapter	Self-hosted workflows for repeatable styles and for users or tiers where hosted cost does not work.	Workflow JSON is versioned in the repo; the adapter submits a workflow, maps node outputs to the contract, and enforces the same timeout and cost fields (GPU time recorded as internal cost).
Direct provider adapter	Reserved for a model only available direct.	Must implement the full contract, including cancel and cost reporting, before it can serve production traffic.

Contract rules

- **Prompts are built, not typed by the model freehand:** scene narration + visual intent + project style block (palette, era, camera language) + negative prompt defaults. The full prompt is stored in the manifest for reproducibility.
- **Seeds and versions:** seed, model ID, and model version are recorded per output. Regenerating a scene reuses the seed unless the user changes the prompt.
- **Aspect discipline:** generation is requested at the timeline aspect (16:9 master; 9:16 is a reframe at render, Section 8) so vertical exports never upscale a vertical-native clip.
- **Failure handling:** a failed or timed-out generation falls back to the visual plan's next preference (usually licensed stock) and marks the scene for review. It never blocks the whole render.
- **Rights:** every generated asset gets a `rights[]` entry naming the provider and its commercial-use terms at generation time.

Stock and the YouTube rule

Default visuals are licensed stock from Pexels / Pixabay, fetched through their APIs with the licence and asset ID stored per clip. Downloading clips from YouTube is **opt-in only**, per project, behind a rights warning the user must confirm: the user asserts they hold the rights or a licence for that material. With opt-in off - the default - no code path in the planner, search, or import can return a YouTube download. User uploads follow the same assertion flow.

7. Motion graphics - parameterised templates, not per-video code

Every graphic is a Remotion component driven by a typed props object stored in the timeline. Changing a number, label, or colour changes props - it never forks the component. MVP ships the eight templates below.

Template	Props	Used when
Bar / line chart	`series[], labels[], unit, highlight_index`	A scene cites comparable numbers with `claim_refs`.
Big number / stat callout	`value, label, sublabel`	One figure carries the scene.
Tier list	`tiers[{name, colour, items[]}]`	Ranking scenes; items are stock thumbnails or text chips.
Comparison split	`left{}, right{}, verdict`	Two options weighed in one scene.
Lower-third / name card	`title, subtitle`	Person, place, or product introduction.
Quote / key line	`text, attribution?`	A line from the narration is the visual.
Progress / timeline	`steps[], current`	Processes, histories, multi-part explainers.
Caption style set	See captions below - captions are a template family, not an overlay afterthought.	Every video.

Template rules

- **Animation:** enter 300-500 ms, exit 200-300 ms, eased, no bounce by default. Motion serves reading order - the highlighted element moves last.
- **Safe areas:** templates render inside 5% title-safe margins and respect the 9:16 reframe zone, so a chart built for 16:9 is still legible when reframed.
- **Data honesty:** chart props are populated from `research.json` claims, not typed by the script model. If the numbers are not in research, the template cannot be selected.
- **Brand kit:** per-project fonts, two accent colours, and logo are props at project level. Templates contain no hard-coded brand values.

Captions

- Captions are generated from `words.json`, grouped into phrases of 3-7 words, timed to speech - never re-transcribed from the final mix.
- Three styles at MVP: clean subtitle, word-highlight (karaoke), and bold centred for Shorts / Reels. Style is a project setting; wording and timing are shared.
- Caption text is editable in the timeline; edits do not change the voice or the transcript record.
- Burned-in captions for vertical exports; a sidecar `.srt` and `.vtt` are always exported with the master, burned-in or not.

8. Timeline, sound, and render

Timeline edit - proposal-based

The timeline holds tracks for video, voice, music, graphics, and captions, all referencing manifest artifacts. Both the user (drag, trim, split in Next.js) and any AI assist act the same way: they submit an edit proposal - a JSON patch against the timeline document - which is validated and applied as a new timeline version. Nothing edits the timeline in place, so undo is version restore and every AI suggestion can be accepted or rejected as a diff.

- Trim and split snap to word boundaries from `words.json` by default. Frame-accurate override is available and marked in the UI.
- Reordering scenes re-flows captions and graphics automatically, because both are anchored to scene IDs, not absolute times.

Sound

- Music: licensed library tracks only, stored with licence entries like any other asset. Default mix ducks music 8-12 dB under voice using voice activity from `words.json`.
- Sound effects: a small licensed set (whoosh, pop, ding) mapped to template events. No per-video hunting through external libraries at MVP.
- Final mix target: -14 LUFS integrated, -1 dBTP true peak, checked at render (Section 9).

Render pipeline - Remotion, then ffmpeg

1. **Compose:** FastAPI freezes a render manifest - timeline version, asset versions, template props, brand kit. A render always names exactly what it rendered.
2. **Preview:** Remotion player in the browser for interactive preview; a 480p proxy render (fast preset) for full-length checks on low bandwidth.
3. **Master render:** Remotion renders the composition on a Celery render worker (1080p, H.264, 30 fps at MVP; 60 fps only where source material justifies it). Long renders are chunked by scene and concatenated, so a failure retries one scene, not the film.
4. **Finish:** ffmpeg normalises loudness, embeds the sidecar captions, writes metadata, and produces the vertical reframe (9:16, subject-aware crop from the visual plan) and square variant where requested.
5. **Publish pack:** master MP4, vertical MP4, 3 thumbnail candidates (frame + template, user-editable text), title and description drafted from the script, chapters from scene boundaries, `.srt` / `.vtt`, and the AI-disclosure note when a cloned or synthetic voice flag is set. Publishing itself is a user action from the pack - the app does not auto-post at MVP.

4K is post-MVP: the composition is resolution-independent, so 4K is a render-profile and pricing change, not a pipeline change. It ships only after 1080p render cost per minute is measured and priced.

9. Quality gates - a render must earn its export

Gates run automatically. A blocking gate stops the export and tells the user what to fix, in the UI, in plain language. Warnings never block, but they are listed in the publish pack so the user exports with open eyes.

Gate	Type	Check
Transcript	Blocking	Cleaned voice re-transcription matches the script word-for-word, allowing only the approved filler removals (Section 5). Caption text matches the same transcript.
Rights	Blocking	Every asset in the render manifest has a `rights[]` entry. Any YouTube-sourced asset requires the project opt-in flag and the user assertion on file.
Audio	Blocking	Final mix within -14 LUFS +/-1 and true peak at or under -1 dBTP. No clipped samples. Voice present in every scene that has narration.
Visual	Blocking	No black frames over 500 ms outside intentional transitions, no missing or placeholder assets, no template rendering with empty props, resolution and frame rate match the render profile.
Text	Blocking	Spell-check on captions and on-screen text in the project language; no unresolved template placeholders; thumbnail text fits its safe area.
Sync	Warning	Caption phrase offset against `words.json` over 300 ms, or a visual change landing more than 1 second from its narration cue.
Content	Warning	Scene states a number with no `claim_ref`; hook scene runs over 5 seconds before the payoff line; total runtime drifts over 15% from target.

Stage-level acceptance tests (build definition of done)

- Research: a seeded topic returns at least 8 sourced claims, and every statistic passes the verbatim source check.
- Cleanup: a test set of 20 recorded clips with known fillers and breaths passes with zero word diffs on re-transcription.
- Generation: killing a worker mid-generation leaves no orphan charges unrecorded and no half-written asset in the manifest.
- Render: a 5-minute reference project renders end-to-end unattended, and a scene-level retry reproduces a bit-identical scene segment.
- Spike exit criterion (day 5-7): one topic goes through all ten stages to a watchable 1080p video with no manual file handling. If it cannot, the MVP scope is cut before the stack is widened.

10. Costs, run order, and the one open decision

All third-party figures below are **estimates / ranges** for planning and credit pricing - they change, and they are verified at procurement.

Cost line	Estimate / range and driver
LLM (research + script)	Estimate: a few US cents to roughly US $0.30 per standard video, driven by research depth and script length.
TTS	Estimate: roughly US $0.02-$0.30 per finished minute depending on provider and voice tier - the widest swing in the budget, hence Section 10's open decision.
Image / video generation	Estimate: roughly US $0.02-$0.10 per image and US $0.10-$1.00+ per short video clip on hosted APIs; ComfyUI shifts this to GPU time instead.
Render + storage	Internal cost: worker compute per rendered minute plus R2 / S3 storage and egress. Measured during the spike; the credit price of a render is set from that measurement, not guessed now.
Stock	Pexels / Pixabay API use is free at MVP volumes under their terms; a paid stock tier is a v1 decision only if the free catalogue proves thin.

Why credits / subscription: research, synthesis, generation, and render all cost money per video. A one-time price would lose money on the heaviest users. Credits map directly onto the per-call costs recorded in the manifest, so a user can always see what a video cost and why.

> **Open decision - primary TTS and voice-clone provider (decide before MVP week 3, when the voice stage is built).** Options: (A) ElevenLabs - strongest cloning quality, highest per-minute estimate; (B) OpenAI TTS - simpler single-vendor billing, no user-voice cloning at the same fidelity, so cloning would need a second provider; (C) self-hosted open TTS on our GPU - lowest marginal cost, highest build and maintenance load, cloning quality unproven on our test set. Default if undecided at the deadline: A for cloning accounts, B for standard voices.

Run order

Spike (5-7 days): manifest + one stage end-to-end (script to rendered scene), cleanup verification loop, one chart template, one generation adapter. **MVP (6-8 weeks):** all ten stages, eight templates, both voice paths, quality gates, credits metering. **v1 (+4-6 weeks):** cloning, vertical-first projects, template additions from usage data, 4K render profile if the cost measurement supports it.